

Ensamble Secuencial de Modelos Predictivos

Aplicado a Tareas de Regresión

Carlos Martín de Prado Barragán
Dept. CCIA
Universidad de Sevilla
Sevilla, España
carmarbar9@alum.us.es

Marco Padilla Gómez
Dept. CCIA
Universidad de Sevilla
Sevilla, España
marpadgom1@alum.us.es

Resumen— El objetivo de este trabajo es aplicar técnicas de aprendizaje supervisado mediante modelos de ensamble secuencial para abordar tareas de regresión en diferentes dominios. Para ello, se ha diseñado un meta-modelo que entrena iterativamente múltiples estimadores base sobre los errores residuales acumulados, mejorando progresivamente la capacidad predictiva del sistema. Este enfoque ha sido evaluado sobre dos conjuntos de datos: *Parkinsons*, orientado al análisis de parámetros biomédicos, y *House Prices*, centrado en la estimación del precio de viviendas. En ambos casos se han utilizado como modelos base *DecisionTreeRegressor* y *LinearRegression*, explorando su rendimiento bajo distintas configuraciones de hiperparámetros. Los resultados reflejan que el ensamble secuencial logra mejorar el rendimiento de los modelos individuales en determinados escenarios, especialmente cuando se emplean estimadores no lineales. Se observa además que *LinearRegression* obtiene mejores resultados en *House Prices* que en *Parkinsons*, lo cual puede estar relacionado con la naturaleza más lineal de las relaciones presentes en el primero, frente a la mayor complejidad y no linealidad del segundo.

Palabras clave— Inteligencia Artificial, Aprendizaje Supervisado, Regresión, Ensamble Secuencial, Modelos Base, Scikit-Learn, Árboles de Decisión, Regresión Lineal, Validación Cruzada.

I. INTRODUCCIÓN

El aprendizaje supervisado es una rama de la inteligencia artificial que se centra en cómo los modelos pueden aprender patrones a partir de datos etiquetados. En particular, en tareas de regresión, el objetivo es predecir valores continuos basándose en un conjunto de variables independientes que describen cada instancia del problema. Estas técnicas han demostrado ser útiles en una amplia gama de aplicaciones, desde el diagnóstico médico hasta la predicción de precios inmobiliarios.

Este proyecto se centra en el desarrollo y evaluación de un modelo de *ensamble secuencial*, diseñado para mejorar la capacidad predictiva de modelos base tradicionales. La motivación principal surge de la necesidad de construir sis-

temas más robustos y precisos mediante la combinación de múltiples predictores individuales. El enfoque seguido consiste en entrenar una secuencia de modelos, donde cada uno intenta corregir los errores cometidos por los anteriores.

La experimentación se ha realizado sobre dos conjuntos de datos muy diferentes: *Parkinson*, que recoge parámetros biomédicos relacionados con el estado motor de pacientes, y *House Prices*, un conjunto clásico para predecir el precio de viviendas en función de características estructurales. En ambos casos se ha aplicado el modelo propuesto utilizando dos algoritmos base: *DecisionTreeRegressor*, que permite capturar relaciones no lineales complejas, y *LinearRegression*, más limitado pero interpretable y eficiente.

La problemática abordada consiste en comprobar si el modelo de ensamble secuencial es capaz de mejorar el rendimiento de estos estimadores individuales en escenarios reales, comparando su comportamiento en dominios de distinta naturaleza y complejidad.

Para cada experimento, se ha seguido una metodología estructurada que incluye: separación de datos en entrenamiento y prueba, entrenamiento del meta-modelo con diferentes configuraciones de hiperparámetros, validación cruzada, y análisis detallado de los resultados mediante métricas estándar y visualizaciones. Todo el desarrollo se ha realizado utilizando *Scikit-Learn* como base tecnológica y en entornos reproducibles mediante notebooks *Jupyter*.

La estructura del documento es la siguiente: En la sección II se describen los métodos empleados y se revisan trabajos relacionados. La sección III detalla la metodología utilizada para la implementación y pruebas del modelo de ensamble. En la sección IV se presentan los resultados obtenidos y su análisis. Finalmente, en la sección V se exponen las conclusiones y propuestas para futuros trabajos.

II. PRELIMINARES

En esta sección se presenta una breve introducción a las técnicas empleadas, así como algunos trabajos relacionados con los modelos de ensamble y los algoritmos de regresión utilizados.

A. Métodos empleados

En este trabajo se han empleado dos estimadores base ampliamente utilizados en problemas de regresión:

Detalles del modelo `DecisionTreeRegressor`

`DecisionTreeRegressor` es un modelo no paramétrico que divide recursivamente el espacio de entrada en regiones homogéneas. Su estructura basada en árboles binarios permite capturar relaciones no lineales complejas entre las variables predictoras y la variable objetivo. Cada nodo del árbol representa una condición de división, y las hojas almacenan el valor medio de las muestras que caen en esa región. Su capacidad para adaptarse a estructuras arbitrarias hace que sea especialmente efectivo en conjuntos de datos con alta variabilidad o relaciones no lineales.

Fig. 1: Descripción del modelo base `DecisionTreeRegressor` utilizado en el ensemble.

Detalles del modelo `LinearRegression`

`LinearRegression` es un modelo lineal que asume una relación directa entre las variables predictoras y la salida esperada. Ajusta los coeficientes de una función lineal minimizando el error cuadrático entre las predicciones y los valores reales. Aunque es rápido y fácil de interpretar, su capacidad para adaptarse a relaciones no lineales es limitada, lo que puede afectar su rendimiento en ciertos dominios complejos.

Fig. 2: Descripción del modelo base `LinearRegression` utilizado en el ensemble.

B. Modelo de ensemble secuencial

Detalles del modelo `SequentialEnsembleRegressor`

`SequentialEnsembleRegressor` es un meta-modelo de regresión que entrena secuencialmente múltiples modelos base sobre subconjuntos aleatorios del conjunto de datos. En cada iteración, el modelo base se ajusta a los residuos del modelo acumulado hasta ese momento. Cada nueva predicción se pondera por una tasa de aprendizaje η y se suma a la predicción total. El muestreo se realiza sin reemplazo sobre una proporción del conjunto de entrenamiento.

Fórmula de actualización acumulada:

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot M_t(X)$$

Donde:

- $\hat{y}^{(t)}$: predicción total después de t iteraciones.
- $M_t(X)$: predicción del modelo base t entrenado sobre un subconjunto aleatorio.
- η : tasa de aprendizaje.

El proceso se repite $n_estimators$ veces, almacenando cada modelo entrenado para usarlo durante la predicción final.

Fig. 3: Funcionamiento del modelo `SequentialEnsembleRegressor` y su fórmula de actualización.

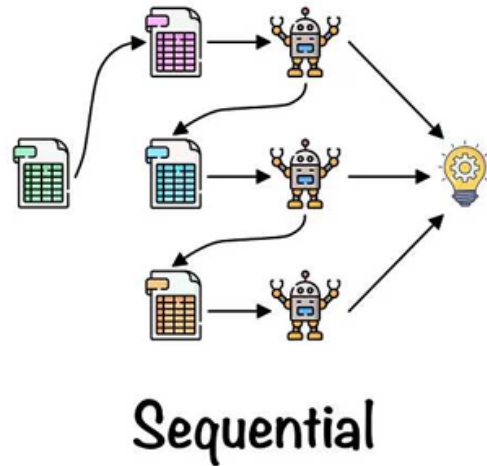


Fig. 4: Representación visual del proceso de ensemble secuencial, donde cada modelo se entrena sobre los errores del anterior.

Las Figuras 1 y 2 resumen los modelos base utilizados, mientras que la Figura 3 muestra el funcionamiento interno del meta-modelo secuencial. Finalmente, la Figura 4 representa visualmente este proceso de forma esquemática.

C. Trabajo relacionado

Diversos trabajos han demostrado la eficacia de los modelos de ensamble secuencial en tareas de predicción, siendo ampliamente utilizados en técnicas como Gradient Boosting. En este proyecto se opta por una implementación manual, lo que permite un control total sobre el proceso y facilita su comprensión didáctica. Además, se ha observado que el tipo de modelo base influye significativamente en el rendimiento del ensamble. Por ejemplo, modelos como `LinearRegression` muestran mejor rendimiento en dominios donde existe una relación aproximadamente lineal entre variables, mientras que estimadores más flexibles como `DecisionTreeRegressor` son capaces de adaptarse a estructuras complejas sin requerir dicha linealidad.

III. METODOLOGÍA

El proyecto se inicia con la implementación del archivo `sequential_ensamble.py`, ubicado en la carpeta `src/`. Este archivo contiene la clase `SequentialEnsembleRegressor`, diseñada para ser totalmente compatible con `Scikit-Learn` y estructurada como un meta-modelo personalizado que entrena iterativamente modelos base.

A continuación se explican en detalle las funciones principales de esta clase:

- **`__init__()`**: Constructor que inicializa los siguientes parámetros:
 - `base_estimator`: clase del modelo base (por ejemplo, `DecisionTreeRegressor`).
 - `n_estimators`: número de modelos que formarán el ensamble.
 - `lr`: tasa de aprendizaje que pondera cada nueva predicción.
 - `sample_size`: proporción de datos (entre 0 y 1) usados para entrenar cada modelo.
 - `est_params`: diccionario con hiperparámetros del estimador base.
 - `random_state`: semilla para asegurar reproducibilidad.
- **`fit(X, y)`**: Realiza el entrenamiento secuencial del ensamble. Para cada iteración:
 - 1) Se calcula el residuo entre los valores reales y las predicciones acumuladas hasta el momento.
 - 2) Se selecciona aleatoriamente un subconjunto de los datos (sin reemplazo).
 - 3) Se entrena un nuevo modelo base sobre esos residuos.
 - 4) Se actualiza la predicción acumulada sumando la predicción del nuevo modelo multiplicada por la tasa de aprendizaje.
- **`predict(X)`**: Aplica todos los modelos entrenados al conjunto de entrada y devuelve la suma ponderada de sus predicciones.

- **`score(X, y)`**: Devuelve el coeficiente de determinación R^2 entre las predicciones obtenidas y los valores reales, usando internamente `r2_score` de `sklearn.metrics`.

Con esta clase definida, se construyó una infraestructura de notebooks modular para analizar el comportamiento del meta-modelo en dos conjuntos de datos: *Parkinson* y *House Prices*, usando dos estimadores base (`DecisionTreeRegressor` y `LinearRegression`).

Cada combinación se trabajó mediante tres notebooks específicos:

Pseudocódigo Notebook Principal

- 1) Cargar el conjunto de datos desde CSV.
- 2) Separar en conjuntos de entrenamiento y prueba.
- 3) Estandarizar las variables predictoras.
- 4) Instanciar el modelo base deseado.
- 5) Crear el meta-modelo `SequentialEnsembleRegressor`.
- 6) Entrenar el meta-modelo sobre los datos de entrenamiento.
- 7) Generar predicciones sobre los datos de prueba.
- 8) Calcular y mostrar métricas como R^2 .
- 9) Comparar visualmente valores reales vs predichos.
- 10) Mostrar ejemplos individuales en forma de tabla.

Fig. 5: Pseudocódigo del notebook principal para entrenamiento y análisis.

Pseudocódigo Validación Cruzada

- 1) Cargar el conjunto de datos desde CSV.
- 2) Estandarizar las variables predictoras.
- 3) Configurar validación cruzada con K particiones.
- 4) Para cada partición:
 - a) Dividir datos en entrenamiento y validación.
 - b) Instanciar el meta-modelo desde cero.
 - c) Entrenar sobre la partición de entrenamiento.
 - d) Calcular R^2 sobre la partición de validación.
- 5) Almacenar métricas por partición.
- 6) Calcular media y desviación típica de R^2 .
- 7) Mostrar resumen de resultados.

Fig. 6: Pseudocódigo del notebook de validación cruzada.

Pseudocódigo Exploración de Hiperparámetros

- 1) Cargar y estandarizar el conjunto de datos.
- 2) Definir listas de valores para los hiperparámetros:
 - Número de estimadores
 - Tasa de aprendizaje
 - Tamaño de muestra
- 3) Para cada combinación de hiperparámetros:
 - a) Instanciar el meta-modelo con dichos valores.
 - b) Realizar validación cruzada sobre los datos.
 - c) Calcular y guardar el R^2 medio.
- 4) Ordenar resultados por puntuación.
- 5) Mostrar las mejores configuraciones.

Fig. 7: Pseudocódigo del notebook de experimentación con hiperparámetros.

La estructura de carpetas del proyecto es la siguiente:

```
ensamble-secuencial-ia/  
  src/  
    sequential_ensamble.py  
  notebooks/  
    parkinsons_dtr/  
      notebook_main.ipynb  
      validacion_cruzada.ipynb  
      experimentos.ipynb  
    parkinsons_lr/  
    house_prices_dtr/  
    house_prices_lr/  
  data/  
    parkinsons.csv  
    house_prices.csv  
  img/  
    portada.png  
  requirements.txt  
  README.md
```

Esta organización facilita la gestión paralela de modelos y conjuntos de datos, así como la reproducción ordenada de cada experimento.

IV. EXPERIMENTACIÓN Y RESULTADOS

Para evaluar el rendimiento del modelo `SequentialEnsembleRegressor`, se diseñaron una serie de experimentos aplicados sobre dos conjuntos de datos distintos: *Parkinsons* y *House Prices*. En cada uno se evaluaron dos modelos base: `DecisionTreeRegressor` y `LinearRegression`. En esta sección se detallan los resultados obtenidos con ambos modelos, en el orden indicado.

A. Modelos con `DecisionTreeRegressor`

1) *Ejecución principal*: Durante la ejecución inicial del meta-modelo secuencial con `DecisionTreeRegressor` como estimador base, se obtuvieron predicciones que reflejan una

mejora significativa respecto al modelo base individual. A continuación se presentan los resultados visuales para ambos conjuntos de datos.

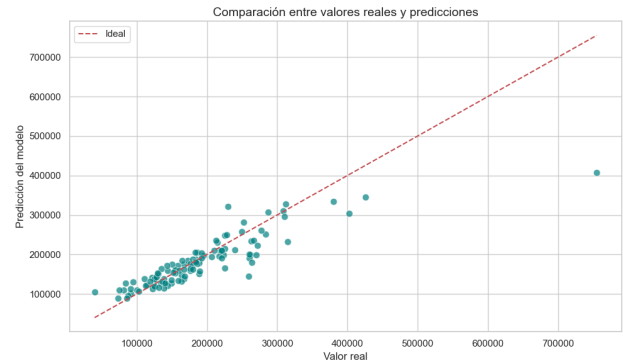


Fig. 8: Predicción vs valor real en House Prices usando `DecisionTreeRegressor`.

En la Figura 8, correspondiente al conjunto *House Prices*, se observa una distribución densa y alineada en torno a la diagonal ideal (línea roja). Esto sugiere que el modelo es capaz de capturar correctamente la tendencia general del problema, con algunas desviaciones en los extremos que podrían estar asociadas a valores atípicos del precio de vivienda.

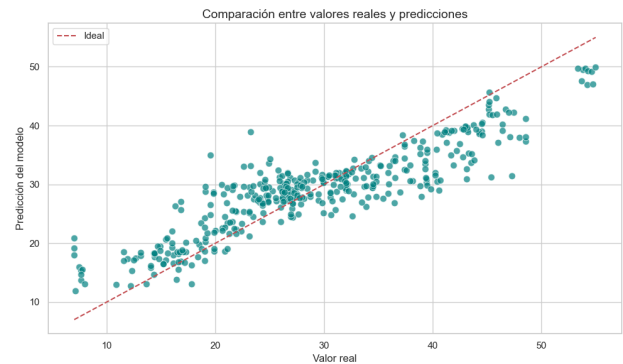


Fig. 9: Predicción vs valor real en Parkinsons usando `DecisionTreeRegressor`.

En la Figura 9, relativa al dataset *Parkinsons*, también se aprecia una alineación razonable con la diagonal ideal. Aunque existe más dispersión que en el caso anterior, los puntos siguen una estructura clara, indicando que el modelo logra adaptarse incluso a relaciones más complejas y no lineales.

2) *Validación cruzada*: Se aplicó validación cruzada con $K = 5$ particiones. En el conjunto *Parkinsons*, el modelo alcanzó un R^2 promedio de 0.78 con una desviación estándar de 0.016, indicando una alta capacidad predictiva y gran estabilidad. En *House Prices*, se obtuvo un R^2 promedio de 0.76 y desviación de 0.032.

3) *Exploración de hiperparámetros*: En los experimentos con `DecisionTreeRegressor` como estimador base, se observó una mejora sustancial del rendimiento en ambos

TABLE I: Resultados de validación cruzada con Decision-TreeRegressor

Dataset	R^2 promedio	Desv. típica
Parkinsons	0.78	0.016
House Prices	0.76	0.032

conjuntos de datos cuando se aplicó el esquema de ensemble secuencial.

Para el conjunto *Parkinsons*, la mejor configuración encontrada fue: $n_estimators = 50$, $lr = 0.2$, $max_depth = 5$, y $sample_size = 0.9$. Con estos parámetros, el modelo alcanzó un R^2 de 0.8865, un MAE (error absoluto medio) de aproximadamente 1.45 y un RMSE (raíz del error cuadrático medio) de 1.97. Este resultado refleja una capacidad predictiva elevada y una baja dispersión de errores, confirmando que el modelo es capaz de capturar la complejidad no lineal del problema. El modelo base individual con la misma profundidad alcanzó solo $R^2 = 0.4939$, lo que refuerza la ventaja del enfoque secuencial.

En el conjunto *House Prices*, la mejor configuración fue: $n_estimators = 50$, $lr = 0.1$, $max_depth = 3$, $sample_size = 0.75$. En este caso, se logró un R^2 de 0.7677, con un MAE de aproximadamente 23,500 euros y un RMSE cercano a 38,000 euros. Considerando el rango de precios de las viviendas, estos errores representan un margen aceptable y útil en aplicaciones reales. De nuevo, el modelo base individual se quedó en $R^2 = 0.4488$, lo que pone de manifiesto el valor del proceso iterativo de corrección de errores.

En resumen, el modelo de ensemble secuencial con árboles de decisión demostró ser especialmente potente tanto en dominios complejos como biomédicos (*Parkinsons*) como en problemas de regresión más estándar (*House Prices*), ofreciendo mejoras claras en precisión y estabilidad frente al modelo individual.

B. Modelos con LinearRegression

1) *Ejecución principal:* Cuando se emplea *LinearRegression* como modelo base, el comportamiento varía significativamente según el dominio de datos. A continuación se muestran las gráficas de comparación para ambos datasets:

La Figura 10 muestra una notable alineación entre predicción y valor real en el conjunto *House Prices*, lo que confirma que el modelo lineal es adecuado para este dominio. La mayoría de los puntos se concentran en torno a la recta ideal, indicando que las relaciones entre variables son predominantemente lineales y fácilmente capturables.

En contraste, la Figura 11 revela una mayor dispersión de las predicciones para el dataset *Parkinsons*, lo cual era esperable dado que la regresión lineal no puede modelar relaciones no lineales complejas. Aunque existe cierta correlación, el error es mayor y la capacidad predictiva es limitada.

2) *Validación cruzada:* Los resultados de validación cruzada reflejan esa diferencia entre dominios. En *Parkinsons*, se alcanzó un R^2 medio de 0.17 con desviación de 0.035, mientras

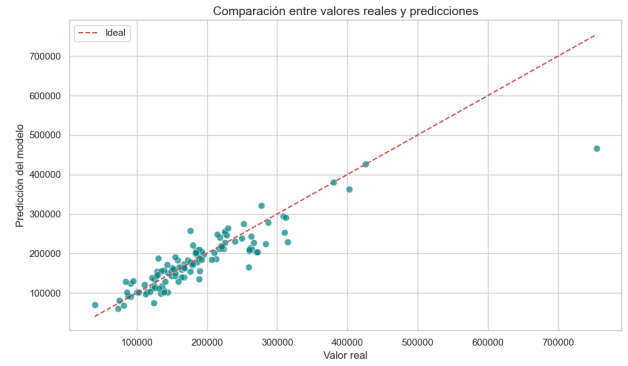


Fig. 10: Predicción vs valor real en House Prices usando LinearRegression.

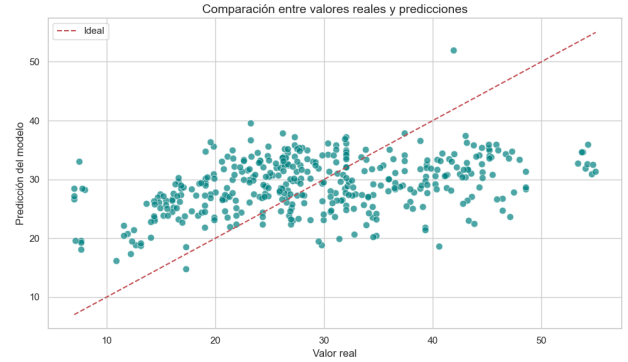


Fig. 11: Predicción vs valor real en Parkinsons usando LinearRegression.

que en *House Prices* se obtuvo un valor promedio de 0.726 con desviación de 0.1251.

TABLE II: Resultados de validación cruzada con LinearRegression

Dataset	R^2 promedio	Desv. típica
Parkinsons	0.17	0.035
House Prices	0.726	0.125

3) *Exploración de hiperparámetros:* En los experimentos con *LinearRegression* como modelo base, también se observó una mejora gracias al ensemble secuencial, aunque más limitada en comparación con los árboles de decisión.

En el conjunto *Parkinsons*, se logró una configuración óptima con $n_estimators = 30$, $lr = 0.2$, y $sample_size = 0.75$. Esta combinación obtuvo un R^2 de 0.1688 (ligeramente por encima del modelo base individual, que alcanzó $R^2 = 0.1680$), un MAE de aproximadamente 4.25 y un RMSE de 5.60. Aunque los resultados siguen siendo bajos, el modelo mostró estabilidad y consistencia en la predicción, y no penalizó el rendimiento respecto al modelo base.

En el conjunto *House Prices*, los resultados fueron más alentadores. La mejor configuración fue: $n_estimators = 200$, $lr = 0.5$, y $sample_size = 0.90$. Se alcanzó un R^2 de 0.7317, un MAE de aproximadamente 26,200 euros y un

RMSE de 41,800 euros. Este resultado representa una mejora real respecto a la regresión lineal individual, que obtuvo R^2 por debajo de 0.65 en la mayoría de configuraciones. Aunque no supera al modelo con árboles, la simplicidad y rapidez del estimador lineal lo convierten en una opción válida para problemas con relaciones predominantemente lineales.

Estos resultados evidencian que, aunque la regresión lineal está limitada por su estructura, el ensamble secuencial le permite corregir errores iterativamente, logrando una mejora sostenida sin sacrificar interpretabilidad ni eficiencia computacional.

C. Síntesis y análisis comparativo

Los resultados experimentales obtenidos permiten realizar un análisis comparativo en dos ejes principales: por tipo de modelo base y por naturaleza del conjunto de datos. A continuación se discuten las observaciones más relevantes.

1) *Comparación entre modelos base:* El uso de `DecisionTreeRegressor` como estimador base ha ofrecido un rendimiento claramente superior al de `LinearRegression` en el conjunto *Parkinsons*, donde se han alcanzado valores de R^2 superiores al 0.88 en configuraciones óptimas, frente al máximo de 0.17 observado con regresión lineal. Esto refleja la capacidad de los árboles de decisión para modelar relaciones no lineales y capturar interacciones complejas entre variables, lo que resulta clave en un dominio biomédico con fuerte componente estructural y variabilidad interindividual.

En el caso del conjunto *House Prices*, aunque `LinearRegression` ha obtenido buenos resultados (hasta $R^2 = 0.73$), `DecisionTreeRegressor` lo supera ligeramente con un R^2 de 0.76 en su mejor configuración. Esta diferencia, si bien menos pronunciada, evidencia que incluso en problemas lineales, la capacidad de modelado no lineal puede ofrecer un pequeño margen de mejora.

Por tanto, se confirma que la flexibilidad del modelo base influye de forma determinante en la capacidad predictiva del meta-modelo secuencial, especialmente cuando los datos presentan patrones complejos o no lineales.

2) *Comparación entre conjuntos de datos:* La elección del conjunto de datos también condiciona fuertemente el rendimiento. En *House Prices*, todos los modelos han mostrado un rendimiento razonablemente alto y estable, con diferencias moderadas entre configuraciones. Esto sugiere que el problema presenta una estructura más regular y lineal, fácilmente aprendible incluso por modelos sencillos.

En contraste, el conjunto *Parkinsons* representa un reto mayor: con regresión lineal, el rendimiento fue bajo y apenas mejoraba respecto a una predicción constante. Sin embargo, al utilizar modelos no lineales como árboles de decisión, el rendimiento se disparó, indicando que las relaciones entre las variables predictoras y la salida son complejas y no triviales.

Esta comparación muestra la importancia de adaptar la complejidad del modelo a la complejidad que ya tiene el propio dominio, evitando el sobreajuste.

Estos hallazgos refuerzan la relevancia del diseño experimental y la adecuación modelo-dominio como pilares fundamentales

en el desarrollo de soluciones de aprendizaje supervisado robustas y generalizables.

V. CONCLUSIONES

Este trabajo ha presentado el desarrollo y la evaluación de un modelo de ensamble secuencial aplicado a tareas de regresión, utilizando como modelos base tanto `DecisionTreeRegressor` como `LinearRegression`. A través de un enfoque iterativo de entrenamiento sobre errores residuales, se ha demostrado que el rendimiento del sistema puede mejorar significativamente frente a modelos individuales, especialmente cuando se seleccionan adecuadamente los hiperparámetros implicados.

Los resultados experimentales reflejan que:

- El ensamble secuencial mejora de forma consistente el rendimiento de los modelos base, especialmente cuando se aplican sobre problemas con relaciones no lineales complejas.
- El uso de `DecisionTreeRegressor` como estimador base ofrece mejores resultados en dominios complejos como *Parkinsons*, superando ampliamente a la regresión lineal.
- En tareas más estructuradas y con relaciones más lineales, como en *House Prices*, tanto los árboles como la regresión lineal se benefician del ensamble, aunque las diferencias entre modelos son menos acusadas.
- Métricas adicionales como el MAE y el RMSE han permitido analizar con mayor profundidad el comportamiento de los modelos, revelando que el ensamble también reduce errores absolutos y dispersión, no solo mejora el R^2 .
- La estructura modular y la compatibilidad del modelo con `Scikit-Learn` facilitan su reutilización, adaptación y extensión futura.

En conjunto, se concluye que el enfoque de ensamble secuencial representa una herramienta eficaz para potenciar la capacidad predictiva de estimadores individuales, siendo especialmente útil cuando se enfrentan tareas de regresión en dominios de alta complejidad.

VI. BIBLIOGRAFÍA

1. J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
2. Scikit-learn Developers, "Gradient Boosting," *scikit-learn documentation*, 2024. [Online]. <https://scikit-learn.org/stable/modules/ensemble.html#gradient-tree-boosting>
3. Página web del curso IA de Ingeniería del Software. <https://www.cs.us.es/docencia/aulavirtual/course/view.php?id=42>
4. Uso de ChatGPT para una mejor comprensión del trabajo, ayuda en errores de código y adaptación de textos a formato LaTeX.